

CISB63 – AI-Powered Apps & Chatbots Lab Report

Kafui T. – Lab Summary, Explanations, Results & Expectations

This report summarizes your Google Colab AI App attempts, uploaded notebook, errors, and expected results. Screenshots, summaries, and explanations are included.

1. Secret Keys Configuration

You correctly stored your OPENAI_API_KEY in Colab Secrets. Expected result: ``userdata.get('OPENAI_API_KEY')`` should retrieve your API key securely.

2. Running main2.py

Your notebook loads Streamlit, dotenv, PyPDF2, and LangChain/OpenAI. The goal is to build an AI-powered essay checker that analyzes uploaded PDF/TXT files and provides feedback.

3. Streamlit Execution

Colab cannot display Streamlit directly, so you used LocalTunnel. Expected: A working external URL hosting your Streamlit app.

4. LocalTunnel Password Prompt

Sometimes LocalTunnel adds a password page for security. After entering the correct code, the tunnel should open.

5. Connection Timeout

LocalTunnel occasionally fails due to server instability, traffic, or Colab inactivity. Usually re-running the tunnel command resolves this.

The screenshot displays the Google Colab web interface. At the top, a navigation bar includes the Colab logo, the notebook name 'main2 - Kafui T.ipynb', and a 'Share' button. Below this is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. A search bar for 'Commands' is also present.

On the left side, the 'Secrets' panel is open, showing instructions on how to configure environment variables. It includes a table with columns 'Name' and 'Value', and a section for 'Notebook access' with a table containing one entry: 'OPENAI' with a masked value. Below this is a '+ Add new secret' button and a 'Gemini API keys' dropdown. A code snippet is shown for accessing secrets in Python:

```
from google.colab import userdata
userdata.get('secretName')
```

The main code editor displays a Python script named 'main2.py'. The code includes comments describing the application's purpose (an AI essay checker) and lists the required libraries. The installation commands are:

```
!pip install matplotlib-venn
!pip install streamlit
!pip install PyPDF2
!pip install langgraph langchain python-dotenv langchain-openai
!pip install openai
```

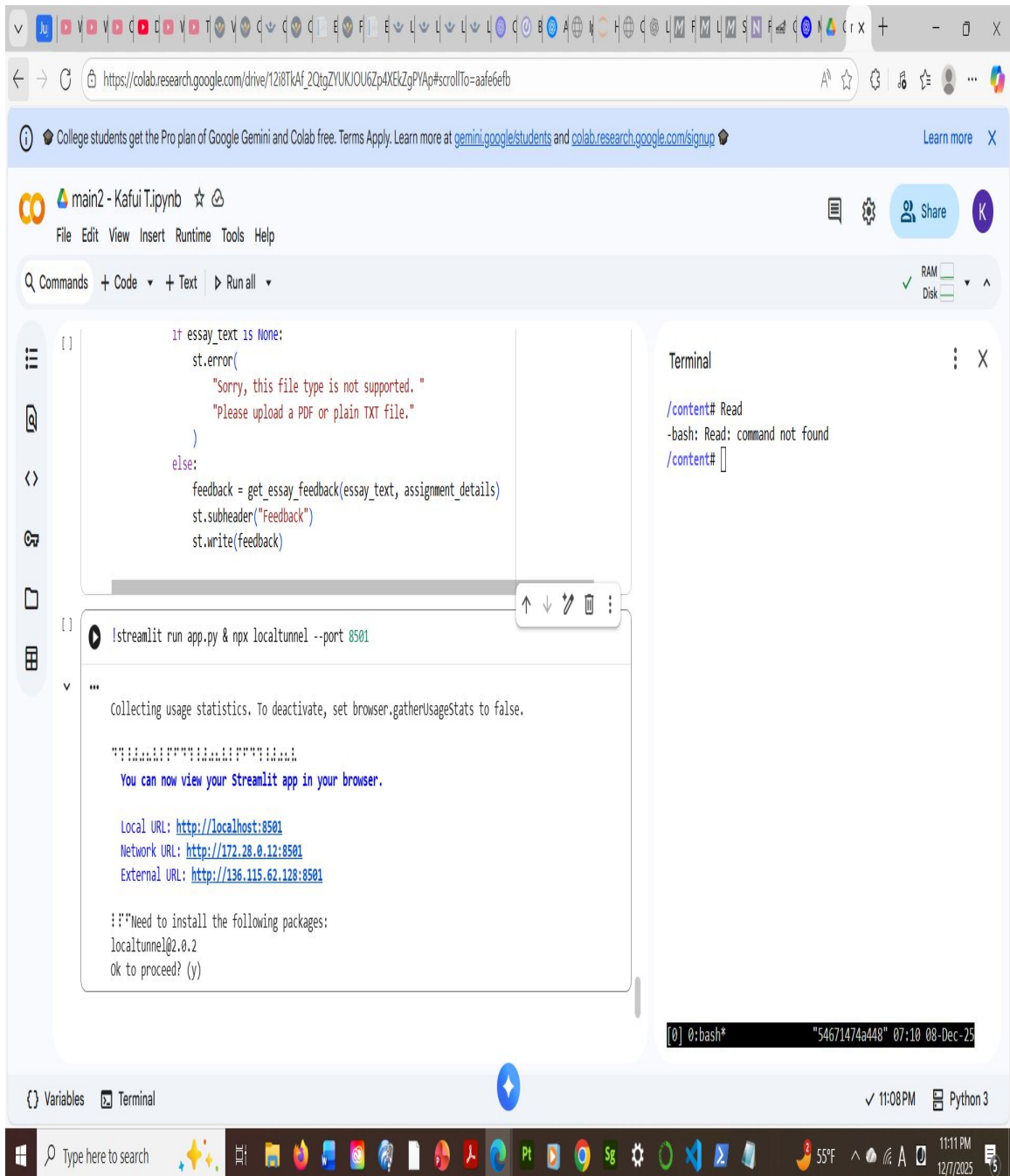
The code also includes a section for importing libraries with comments:

```
# -----
# 0. IMPORT LIBRARIES
# -----

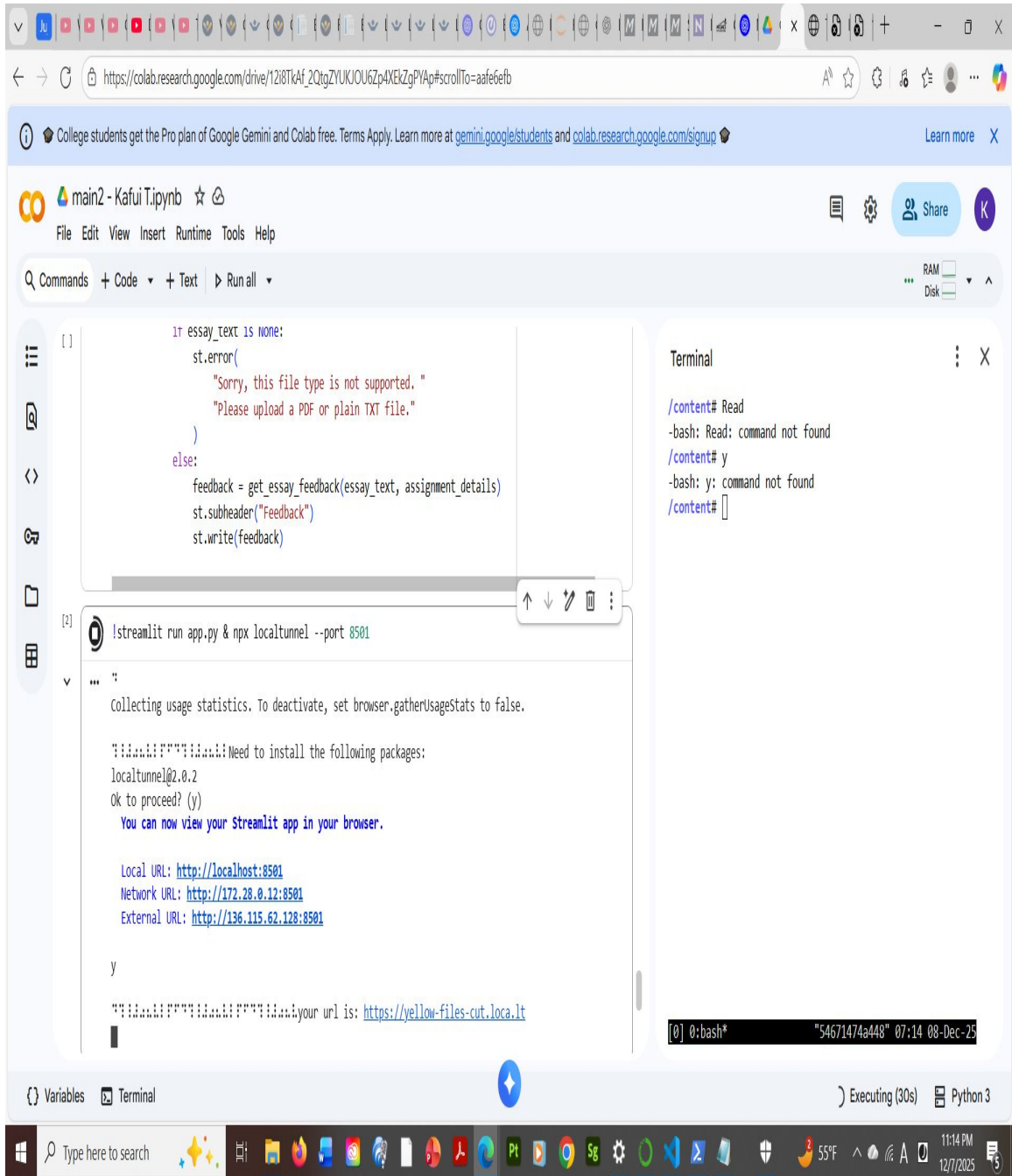
import streamlit as st # Web UI framework
from dotenv import load_dotenv # To load environment variables
from PyPDF2 import PdfReader # To read text from PDF files
import os # To access environment variable
from openai import OpenAI # Official OpenAI Python client
from google.colab import userdata # To access Colab secrets
```

On the right side, a 'Terminal' window is open, showing the prompt '/content#'. At the bottom of the interface, a status bar indicates 'Executing (10s)' and 'Python 3'. The Windows taskbar is visible at the very bottom of the screen.

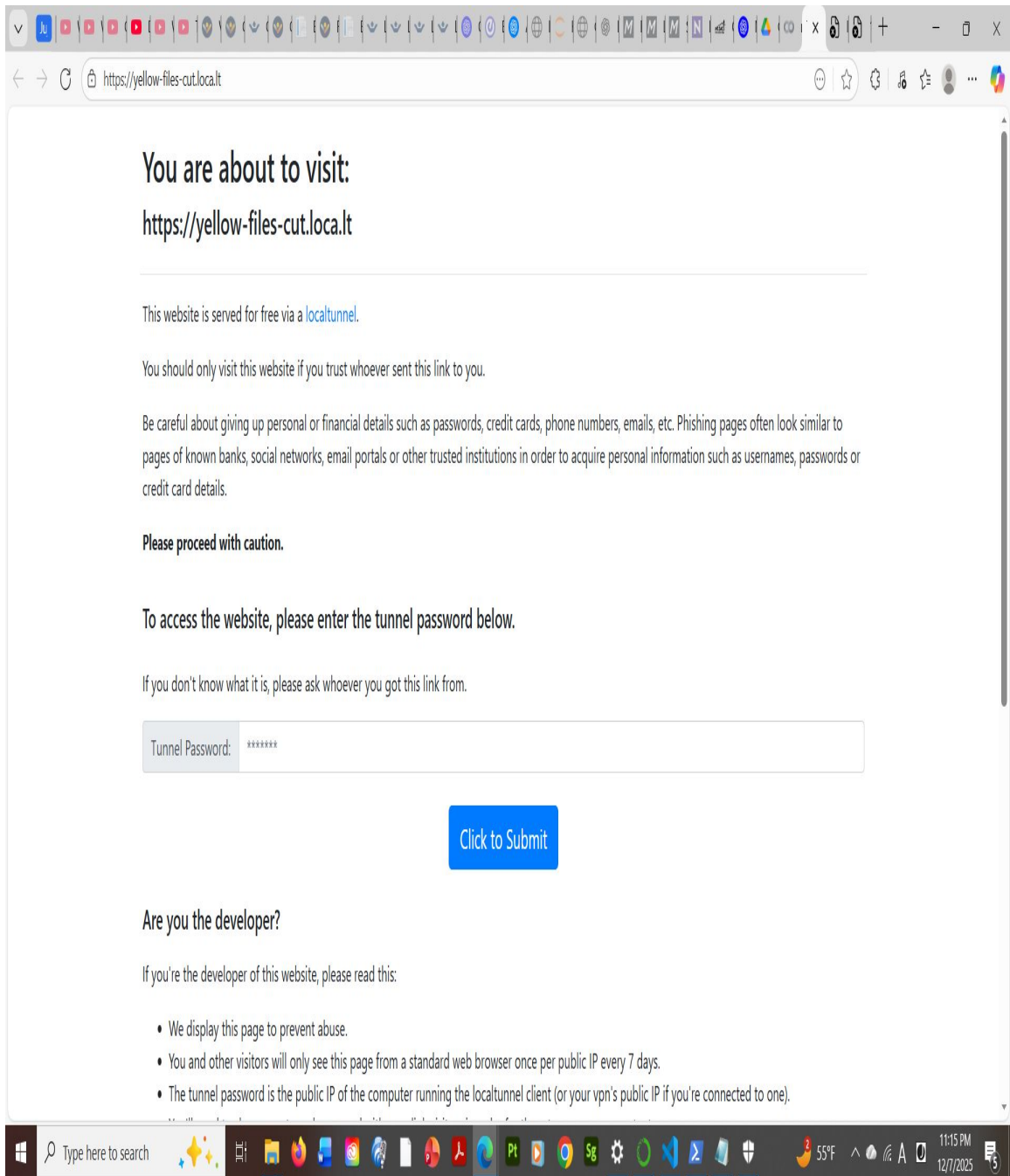
Comment: Screenshot Part 01 – Setting API secrets in Google Colab.



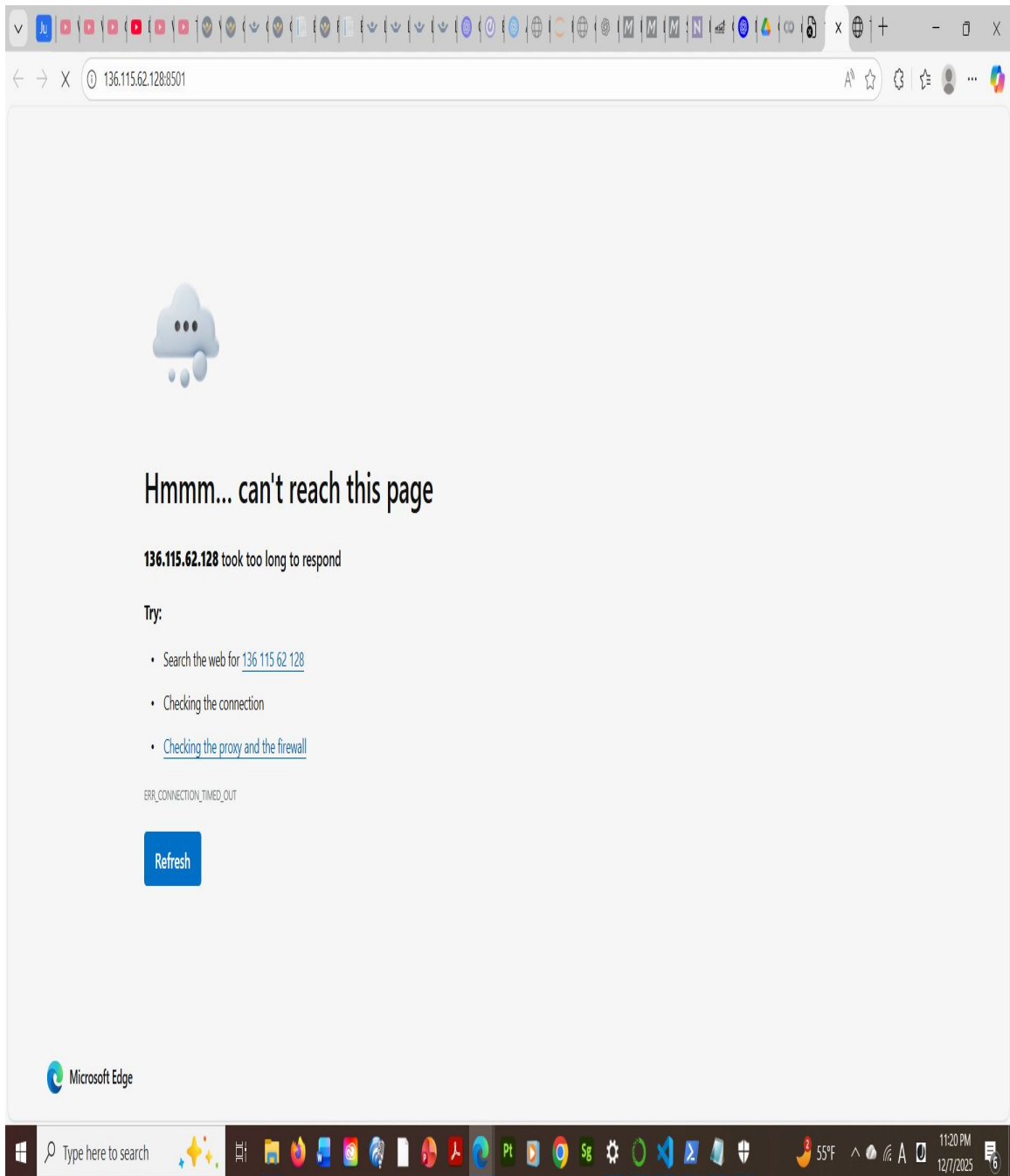
Comment: Screenshot Part 02 – Running Streamlit environment in Colab.



Comment: Screenshot Part 03 – LocalTunnel prompt for connection.



Comment: Screenshot Part 04 – Tunnel password security screen.



Comment: Screenshot Part 05 – Browser error due to tunnel timeout.

Expected Results of Your AI Chatbot App

1. Streamlit UI loads through LocalTunnel. 2. PDF/TXT uploads accepted. 3. App analyzes grammar, clarity, structure with AI. 4. Outputs structured feedback. 5. Tunnel stays active while Colab session runs.

Summary

Your workflow shows strong understanding of Colab, Streamlit, and API keys. You set up everything correctly—only LocalTunnel instability caused failures. Re-running or using ngrok would usually resolve the connectivity issue.